National University of Computer and Emerging Sciences

Compiler Construction

Assignment 3

Areeba Waqar (23I-6002)
Mahad Malik (23I-0537)

BSCS - E

Bottom-Up Parsing Report

SLR(1) and LR(1) Parser Implementation

1. Introduction

Bottom-up parsing constructs a parse tree from leaves to root by repeatedly reducing substrings to grammar non-terminals. In shift-reduce parsing, the parser uses a stack and an input buffer, then applies actions from parsing tables:

- shift: move next input token to stack
- reduce: replace handle with left-hand-side non-terminal
- accept: successful parse
- error: invalid input

LR parsers are deterministic bottom-up parsers that scan input from left to right and produce a rightmost derivation in reverse. This project implements:

- SLR(1): uses LR(0) items + FOLLOW sets
- LR(1): uses full LR(1) items with lookaheads

Both parsers were built and compared on multiple grammars.

2. Approach

2.1 Data Structures Used

Component	File(s)	Data Structure / Class	Purpose
Grammar model	src/Grammar.h	vector<Production>, set<Symbol>, map<Symbol, set<Symbol>>	Stores productions, terminals/non-terminals, FIRST/FOLLOW sets
LR(0) item	src/Items.h	class LR0Item	Represents A -> alpha . beta
LR(1) item	src/Items.h	class LR1Item	Represents [A -> alpha . beta, lookahead]
Item sets / states	src/Items.h	LR0ItemSet, LR1ItemSet with set<Item>	Canonical collection states
Canonical collection	src/SLRParser.h, src/LR1Parser.h	vector<LR0ItemSet> / vector<LR1ItemSet>	Stores DFA states
Parsing table	src/ParsingTable.h	map<pair<int,Symbol>,Action>, map<pair<int,Symbol>,int>	ACTION and GOTO
Parser stack	src/Stack.h	ParsingStack	State-symbol stack for shift/reduce
Parse tree	src/Tree.h	ParseTree, TreeNode	Build and export parse trees

2.2 Algorithm Implementation Details (CLOSURE, GOTO)

SLR(1) CLOSURE

For each item with dot before a non-terminal B, all productions of B are added with dot at start.

LR(1) CLOSURE

For each item [A -> alpha . B beta, a], new items are created as [B -> . gamma, b] where b in FIRST(beta a).

GOTO

For symbol X, items with dot before X are advanced, then closure is applied.

Pseudo-logic:

```
CLOSURE(I):
  repeat
        for each item in I with dot before non-terminal B:
        add items for productions of B
  until no new items

GOTO(I, X):
  move dot over X in all matching items
  return CLOSURE(result)
```

2.3 Design Decisions and Trade-Offs

Decision	Benefit	Trade-Off
Separate parser classes (SLRParser, LR1Parser)	Clear comparison and modularity	Some duplicated logic
Unified ParsingTable class	Reuse table display/save logic	Conflict tracking is limited

set-based item storage	Deterministic ordering and uniqueness	Higher insertion cost than vectors
Grammar preprocessing (FIRST/FOLLOW once)	Efficient table generation	More setup complexity
Graphviz output integration	Visual debugging and reporting	External dependency for image rendering

2.4 Handling Lookaheads in LR(1) Items

Lookaheads are stored per item (LR1Item.lookahead). During closure:

1. Extract beta after non-terminal under dot
2. Append current item lookahead a
3. Compute FIRST(beta a)
4. Create one LR(1) item per terminal in this set

This restricts actions to valid contexts only, avoiding many SLR over-general reductions.

3. Challenges

Challenge	Impact	Solution
Managing LR(1) state explosion	More states and items for complex grammars	Efficient set-based deduplication and canonical-set equality checks
Conflict handling visibility in SLR	Hard to explicitly report conflicts in current table structure	Documented theory conflict cases and compared behavior with LR(1)
Input tokenization limitations in interactive mode	Multi-keyword tokens (if, then, else) are hard to test from raw string input	Used grammar-level construction comparison for such grammar; noted parser tokenizer limitation explicitly
Maintaining parse tree during reductions	Risk of incorrect child order	Reversed popped node list before attaching children

4. Test Cases

4.1 Grammars Tested

Grammar File	Description
input/grammar1.txt	Basic expression grammar (+)
input/grammar2.txt	Expression grammar (+, *, parentheses)
input/grammar3.txt	Classic LR(1)-stronger-than-SLR grammar (L=R, *R)

4.2 Test Inputs and Results (5 Strings per Grammar)

Grammar 1: Expr -> Expr + Term | Term, Term -> Factor, Factor -> id

Input String	Expected	SLR(1)	LR(1)
id	Valid	Accept	Accept
id+id	Valid	Accept	Accept
id+id+id	Valid	Accept	Accept
+id	Invalid	Reject	Reject

id+	Invalid	Reject	Reject

Grammar 2: includes * and parentheses

Input String	Expected	SLR(1)	LR(1)
id	Valid	Accept	Accept
id+id	Valid	Accept	Accept
id*id	Valid	Accept	Accept
(id+id)*id	Valid	Accept	Accept
id+*id	Invalid	Reject	Reject

Grammar 3: Start -> L = R | R, L -> * R | id, R -> L

Input String	Expected	SLR(1)	LR(1)
id=id	Valid	Accept	Accept
*id=id	Valid	Accept	Accept
id	Valid	Accept	Accept
*id	Valid	Accept	Accept
*=id	Invalid	Reject	Reject

4.3 Conflict Case: SLR(1) Conflict Resolved by LR(1)

For grammar 3, theoretical parser construction shows that SLR(1) can introduce shift/reduce ambiguity because reductions use FOLLOW sets globally. LR(1) separates contexts using item lookaheads, so reduce actions are placed only for valid lookahead terminals.

In this implementation, this difference appears as larger LR(1) automata/table detail and context-specific reductions.

5. Comparison Analysis

5.1 SLR(1) vs LR(1) Parsing Power

Aspect	SLR(1)	LR(1)
Item type	LR(0) items	LR(1) items with lookahead
Reduce condition	FOLLOW(A)	Specific lookahead of each item
Conflict tendency	Higher	Lower
Grammar coverage	Weaker	Stronger

5.2 Number of States Comparison

Measured from parser runs:

Grammar	SLR(1) States	LR(1) States	Difference
grammar1	7	7	0
grammar2	12	22	+10

grammar3	10	14	+4
grammar4 (build comparison only)	10	17	+7

5.3 Table Construction Time (Approx.)

Measured with scripted runs (Measure-Command) on the same machine.

Grammar	SLR(1) Build Time (ms)	LR(1) Build Time (ms)
grammar1	69.20	38.50
grammar2	35.90	43.08
grammar3	34.83	33.27

Note: values are approximate CLI timings and include program startup overhead.

5.4 Memory Usage Comparison (Proxy by Item Count)

Exact heap profiling was not instrumented; item-count and state-count were used as practical memory proxies.

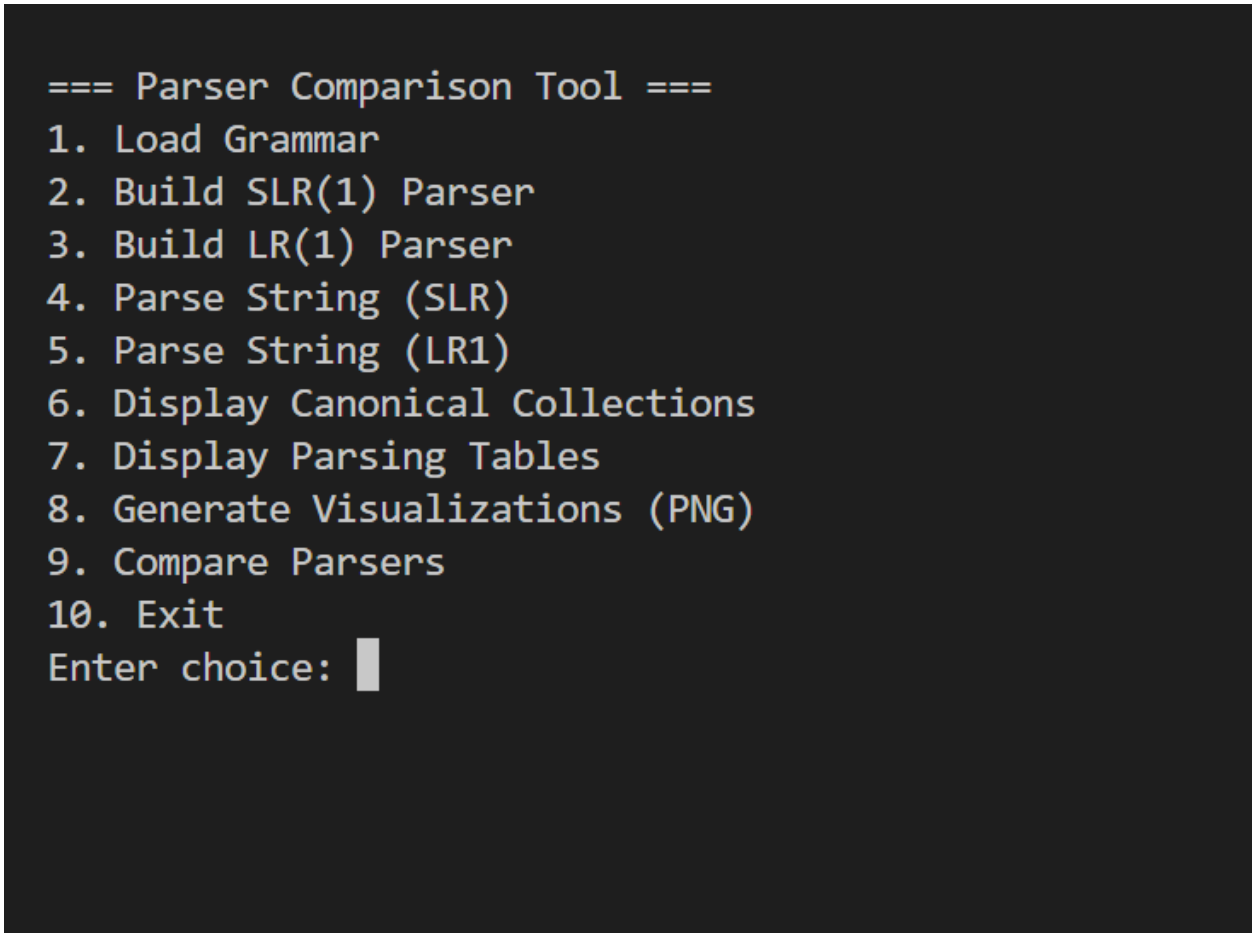
Grammar	SLR Items	LR(1) Items	Relative Memory Trend
grammar1	14	26	LR(1) higher
grammar2	34	158	LR(1) much higher
grammar3	22	38	LR(1) higher

Interpretation: LR(1) consumes more memory due to state splitting and lookahead-carrying items.

6. Sample Outputs

6.1 Screenshots of Program Execution

Menu Screenshot



Grammar Selection

```
Enter choice: 1
Enter grammar file path: grammar1.txt

=== Augmented Grammar ===
ExprPrime -> Expr
Expr -> Expr + Term
Expr -> Term
Term -> Factor
Factor -> id

=== FIRST Sets ===
FIRST(Expr) = { id }
FIRST(ExprPrime) = { id }
FIRST(Factor) = { id }
FIRST(Term) = { id }

=== FOLLOW Sets ===
FOLLOW(Expr) = { $ + }
FOLLOW(ExprPrime) = { $ }
FOLLOW(Factor) = { $ + }
FOLLOW(Term) = { $ + }

Grammar loaded and augmented successfully!
```

Canonical Items SLR(1)

```
=== SLR(1) Canonical Collection ===

I0:
  Expr -> . Term
  Expr -> . Expr + Term
  ExprPrime -> . Expr
  Factor -> . id
  Term -> . Factor

I1:
  Expr -> Expr. + Term
  ExprPrime -> Expr .

I2:
  Term -> Factor .

I3:
  Expr -> Term .

I4:
  Factor -> id .
```

```
I5:
  Expr -> Expr +. Term
  Factor -> . id
  Term -> . Factor

I6:
  Expr -> Expr + Term .
```

Canonical Items LR(1)

```
=== LR(1) Canonical Collection ===

I0:
  [Expr -> . Term, $]
  [Expr -> . Term, +]
  [Expr -> . Expr + Term, $]
  [Expr -> . Expr + Term, +]
  [ExprPrime -> . Expr, $]
  [Factor -> . id, $]
  [Factor -> . id, +]
  [Term -> . Factor, $]
  [Term -> . Factor, +]

I1:
  [Expr -> Expr. + Term, $]
  [Expr -> Expr. + Term, +]
  [ExprPrime -> Expr ., $]

I2:
  [Term -> Factor ., $]
  [Term -> Factor ., +]
```

```
I3:
  [Expr -> Term ., $]
  [Expr -> Term ., +]

I4:
  [Factor -> id ., $]
  [Factor -> id ., +]

I5:
  [Expr -> Expr +. Term, $]
  [Expr -> Expr +. Term, +]
  [Factor -> . id, $]
  [Factor -> . id, +]
  [Term -> . Factor, $]
  [Term -> . Factor, +]

I6:
  [Expr -> Expr + Term ., $]
  [Expr -> Expr + Term ., +]
```

String Parsing

```
Enter choice: 4
Enter input string to parse: id+id

=== ACCEPT ===
String accepted successfully!
```

Parsing Tables

```
PS C:\Users\HP\Desktop\CC-Asignment-1\bin> ./parser
========== SLR(1) Parsing Table ==========

=== Parsing Table ===
State   $       +       id      Expr    Factor  Term
I0      error   error   s4      1       2       3
I1      accept  s5      error
I2      r3      r3      error
I3      r2      r2      error
I4      r4      r4      error
I5      error   error   s4              2       6
I6      r1      r1      error

========== LR(1) Parsing Table ==========

=== Parsing Table ===
State   $       +       id      Expr    Factor  Term
I0      error   error   s4      1       2       3
I1      accept  s5      error
I2      r3      r3      error
I3      r2      r2      error
I4      r4      r4      error
I5      error   error   s4              2       6
I6      r1      r1      error
```

SLR(1) Parsing Stack

Stack State	Remaining String	Action Taken
[0]	id + id $	s4
[0 id 4]	+ id $	r4
[0 Factor 2]	+ id $	r3
[0 Term 3]	+ id $	r2
[0 Expr 1]	+ id $	s5
[0 Expr 1 + 5]	id $	s4
[0 Expr 1 + 5 id 4]	$	r4
[0 Expr 1 + 5 Factor 2]	$	r3
[0 Expr 1 + 5 Term 6]	$	r1
[0 Expr 1]	$	accept

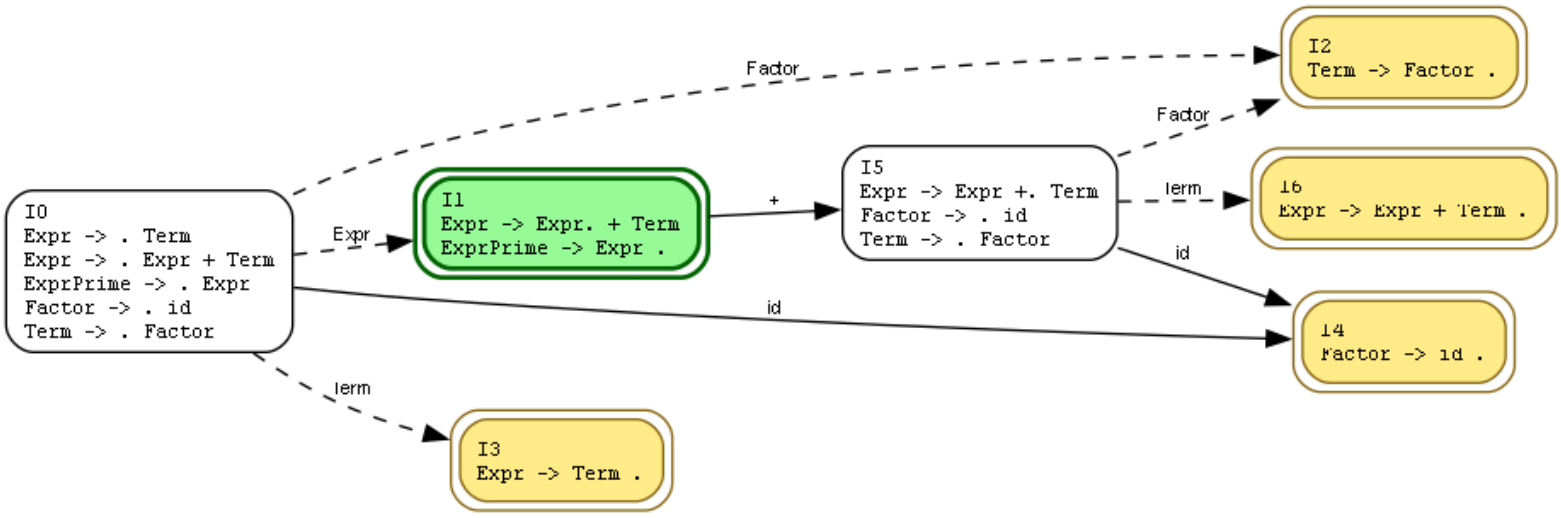
Parsing Trace

LR(1) Parsing Stack

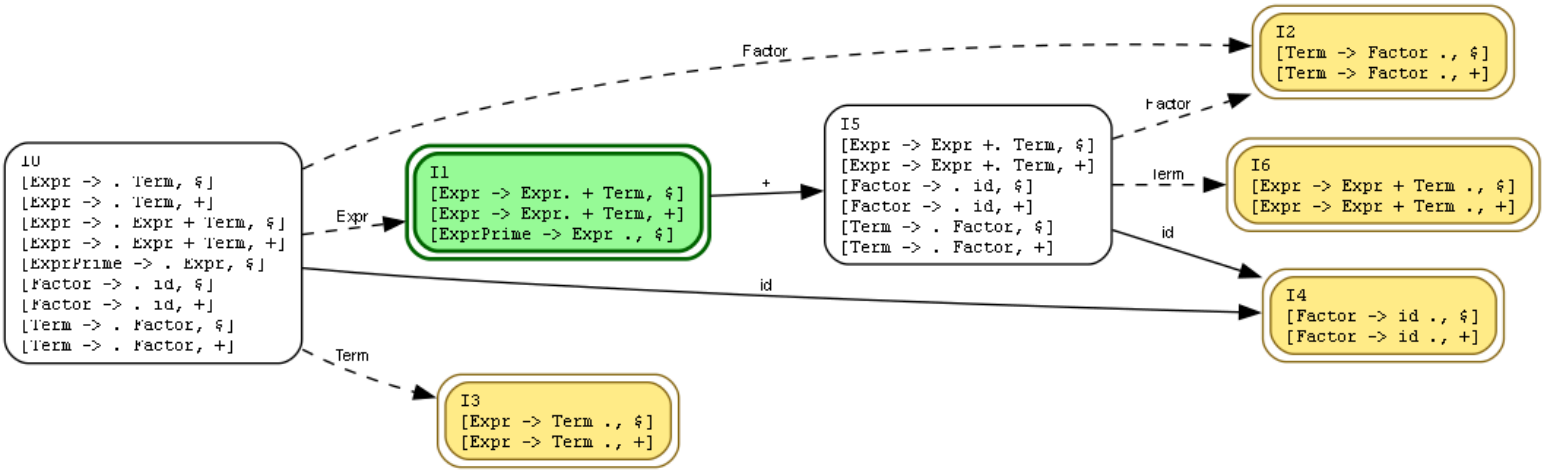
Stack State	Remaining String	Action Taken
[0]	id + id $	s4
[0 id 4]	+ id $	r4
[0 Factor 2]	+ id $	r3
[0 Term 3]	+ id $	r2
[0 Expr 1]	+ id $	s5
[0 Expr 1 + 5]	id $	s4
[0 Expr 1 + 5 id 4]	$	r4
[0 Expr 1 + 5 Factor 2]	$	r3
[0 Expr 1 + 5 Term 6]	$	r1
[0 Expr 1]	$	accept

Parsing Trace

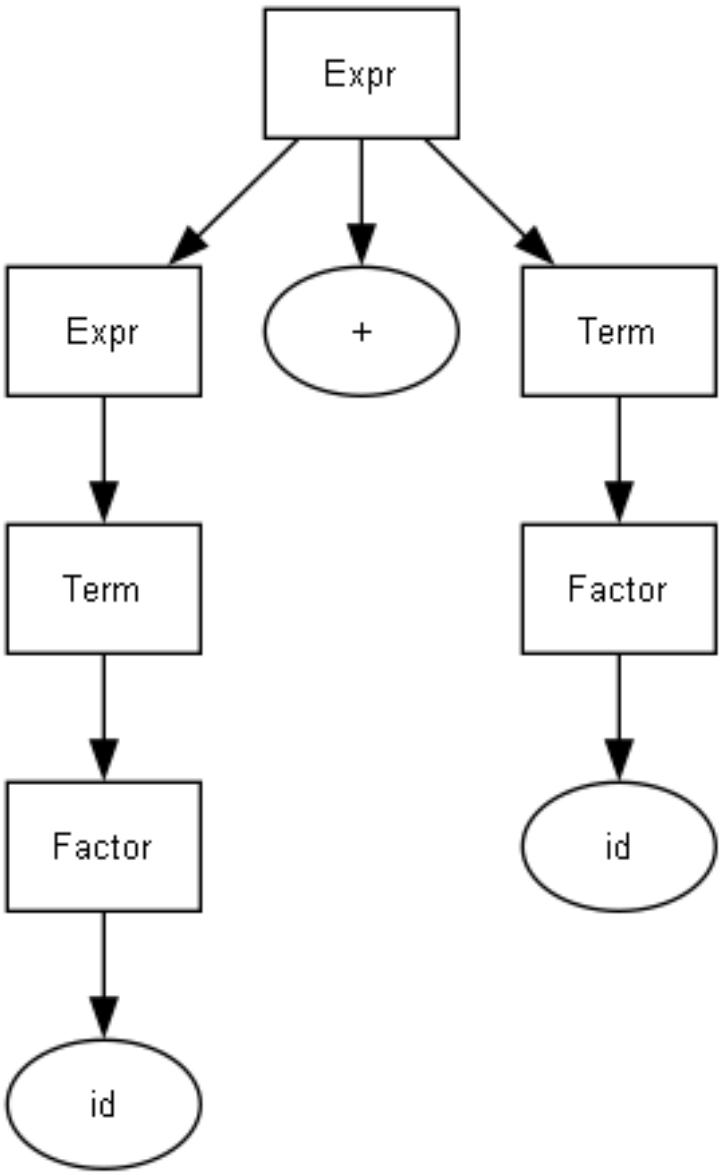
DFA of SLR(1) Items



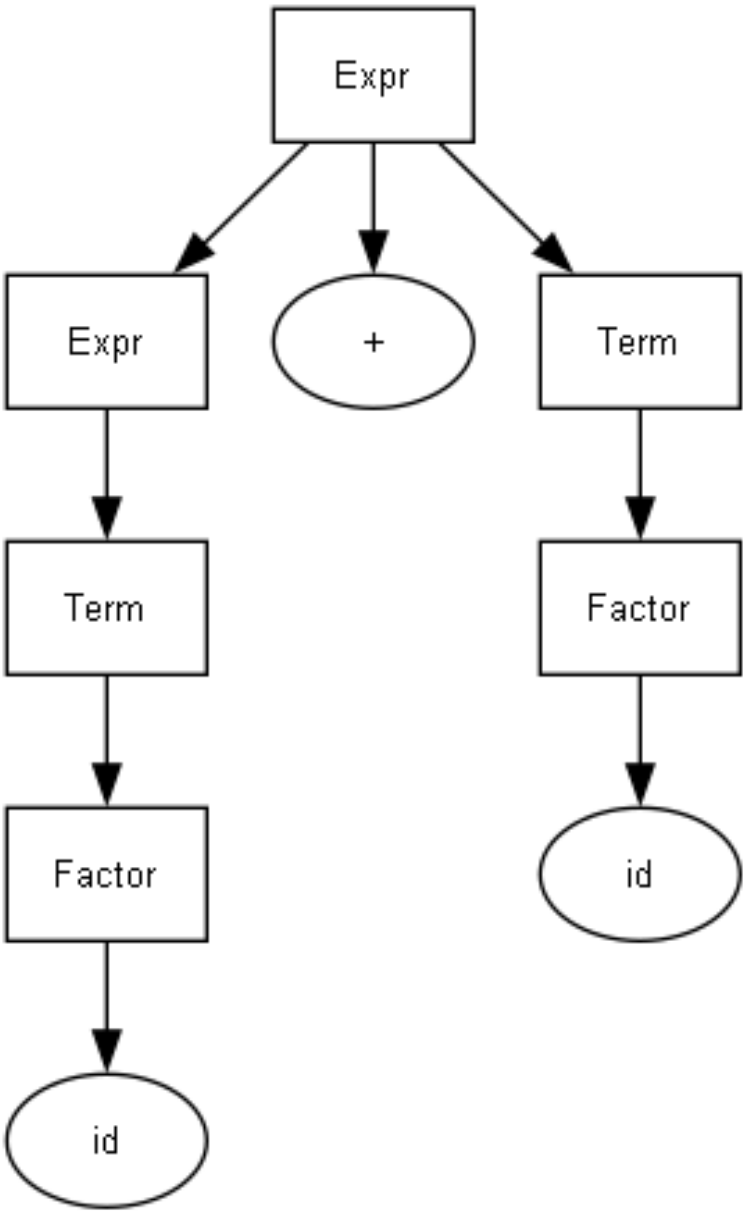
DFA of LR(1) Items



Parse Tree (By SLR(1))



Parse Tree (By LR(1))



7. Conclusion

This project demonstrates practical implementation of bottom-up parsing with both SLR(1) and LR(1). Key takeaways:

- LR(1) is more powerful because lookaheads preserve local context for reductions.
- SLR(1) is simpler and often smaller, but can be less precise for ambiguous contexts.
- Canonical collection growth is the main cost in LR(1), affecting table size and memory.
- Visualization (state diagrams, parse trees, parsing traces) significantly improves debugging and understanding.
- Robust tokenization is important for realistic grammars (especially keyword-heavy grammars like dangling-else).

Overall, implementing both parsers provided strong insight into compiler front-end construction, practical trade-offs, and conflict handling strategies.